lz_codec — Internal Developer Documentation

KKO 2025/2026

Martin Pribylina (xpriby19)

May 2026

Contents

1 Introduction

1.1 Project Overview

`lz_codec` is a command-line lossless compressor for raw 8-bit grayscale images, implemented in C++17 and built with GNU Make. It was written by Martin Pribylina (xpriby19) as a course project for KKO 2025/2026 at FIT VUT Brno.

The tool compresses and decompresses files using the LZSS sliding-window dictionary algorithm, optionally combined with a first-order pixel difference model and per-block adaptive scan direction selection. All information needed for decompression is stored inside the compressed file; no flags need to be passed to the decompressor beyond the input and output file paths.

1.2 Assignment Constraints

The binary must support the following CLI flags:

Flag	Meaning
`-c`	Compress mode
`-d`	Decompress mode
`-i` *file*	Input file path
`-o` *file*	Output file path
`-w` *N*	Image width in pixels (required for compression)
`-m`	Enable pixel difference model (compression only)
`-a`	Enable adaptive 16×16 block scanning (compression only)

Key constraints from the assignment:

- **Self-contained format**: all decompression parameters (`-m`, `-a`, `-w`) are stored in the compressed file header. The decompressor never needs these flags.
- **No external compression libraries**: the LZSS encoder/decoder is implemented from scratch.
- **C++17**: compiled with GCC on `merlin.fit.vutbr.cz` (x86-64 Linux).
- **Portable integers**: `uint8_t`, `uint16_t`, `uint32_t` used throughout.

1.3 Supported Compression Modes

The tool supports four combinations of the two optional pre-processing flags:

Mode	Scan	Model
1 (default)	Static horizontal	Off
2	Static horizontal	Delta model on
3	Adaptive 16×16	Off
4	Adaptive 16×16	Delta model on

In all modes the core LZSS algorithm performs the actual byte-count reduction. The scan order and delta model are pre-processing steps that rearrange or transform the pixel data to increase the repetition density that LZSS exploits.

2 Architecture

2.1 Component Map

The codebase consists of seven source modules, each with a single well-defined responsibility:

Module	Responsibility
`args (.hpp / .cpp)`	CLI argument parsing; populates `ParsedArgs`
`bitio (.hpp only)`	Bit-level read/write abstraction over byte streams
`lzss (.hpp / .cpp)`	LZSS sliding-window encoder and decoder
`model (.hpp / .cpp)`	Forward and inverse first-order delta transform
`scanner (.hpp / .cpp)`	Image-to-stream reordering and block decomposition
`codec (.hpp / .cpp)`	Top-level compress/decompress orchestration + file format
`main (.cpp only)`	Entry point; opens streams, dispatches to codec

2.2 Compression Data Flow

The following steps occur during a compression call:

1. `main.cpp` calls `parse_arguments(argc, argv)` → `ParsedArgs`.
2. Opens `args.infile` as a binary `std::ifstream` and `args.outfile` as a binary `std::ofstream`.
3. Calls `compress(in, out, args.width, args.use_model, args.adaptive_scan)`.
4. `codec.cpp`: reads all bytes from `in` into `vector<uint8_t> pixels`. Validates that `pixels.size() % width == 0`.
5. Writes the LZKO file header (magic, width, height, flags, block_size_log2, original size).
6. **Static path** (no `-a`):
   (a) `scan_horizontal(pixels, w, h)` → 1-D byte stream (trivial copy).
   (b) If `-m`: `forward_model(data)` transforms in-place.
   (c) `lzss_encode(data, BitWriter(out))`; then `bw.flush()`.
7. **Adaptive path** (`-a`), two passes:
   (a) **Pass 1 — metadata**: `split_into_blocks(pixels, w, h, 16)` → `blocks`. For each block: compute SAD in both directions; choose lower-SAD direction as `scan_dir`; scan; optionally `forward_model`; `lzss_encode_to_buffer` to get compressed size; set `meta.compressed` = 1 if compressed < raw.
   (b) Write `num_blocks` and packed 2-bit-per-block metadata to output.
   (c) **Pass 2 — data**: repeat scan + model + encode for each block; write `uint32_t stored_size` followed by the block bytes.

2.3 Decompression Data Flow

The following steps occur during a decompression call:

1. `main.cpp` calls `decompress(in, out)`.
2. `codec.cpp`: reads and validates magic bytes; returns error code 1 on mismatch.
3. Reads header fields: `width`, `height`, `flags`, `block_size_log2`, `original_size`. Reconstructs `use_model` and `adaptive` from `flags`.
4. **Static path**:
   (a) `lzss_decode(BitReader(in), data, original_size)`.
   (b) If `use_model`: `inverse_model(data)`.
   (c) Write `data` to `out`.
5. **Adaptive path**:
   (a) Read `num_blocks` and packed block metadata; unpack into `scan_dirs[]` and `comp_flags[]`.
   (b) Allocate full image buffer `vector<uint8_t> image(width * height)`.
   (c) For each block index `idx`: read `stored_size` + raw bytes; if `comp_flags[idx]`: wrap in `istringstream` + `BitReader` + `lzss_decode`; else use raw bytes directly.
   (d) If `use_model`: `inverse_model(block_data)`.
   (e) `unscan_horizontal` or `unscan_vertical` depending on `scan_dirs[idx]`.
   (f) Copy block pixels back into correct position in `image` buffer.
   (g) After all blocks: write `image` to `out`.

2.4 Dependency Graph

Module	Depends on
args	(none)
bitio	(none)
model	(none)
scanner	(none)
lzss	bitio
codec	bitio, lzss, model, scanner
main	args, codec

2.5 File Format

Every compressed file begins with the LZKO header:

Offset	Size	Field
0	4 B	Magic bytes: `0x4C 0x5A 0x4B 0x4F` (“LZKO”)
4	2 B	`uint16_t` image width (little-endian)
6	2 B	`uint16_t` image height (little-endian)
8	1 B	Flags: bit 0 = use_model, bit 1 = adaptive
9	1 B	`block_size_log2` ($= 4 \rightarrow 2^4 = 16$ pixels)
10	4 B	`uint32_t` original pixel count (little-endian)
14	...	Compressed data (static) *or* block section (adaptive)

In adaptive mode, the data section is structured as:

- `uint32_t num_blocks` — total number of 16×16 blocks.
- Bit-packed metadata: 2 bits per block, MSB-first, padded to a byte boundary. Bit 0 of each pair = `scan_dir`; bit 1 = `compressed`.
- Per-block data: `uint32_t stored_size` followed by `stored_size` bytes (LZSS-compressed or raw).

3 CLI Arguments Module

3.1 Purpose

The args module is the single point of CLI parsing. It decouples `main.cpp` from raw `argc`/`argv` handling and makes the parsed configuration available to the rest of the program through a plain data struct. All validation that can be done at the CLI level (e.g., checking that required flags are present) happens here, so the codec functions can assume a coherent input.

3.2 Files

- `include/args.hpp` — `ParsedArgs` struct and `parse_arguments` declaration.
- `src/args.cpp` — implementation using `argparse.hpp`.
- `include/argparse.hpp` — third-party header-only argument parsing library (bundled in-tree; see Section 3.5).

3.3 ParsedArgs Struct

```
1  struct ParsedArgs {
2      bool        decompress    = false;
3      std::string infile;
4      std::string outfile;
5
6      // Compression-only parameters.
7      // During decompression these are IGNORED -- loaded from the
           file header.
8      bool use_model    = false;
9      bool adaptive_scan = false;
10     int  width        = -1;
```

```
11 };
```

Field documentation:

Field	Meaning
`decompress`	`true` when `-d` flag is present; `false` for `-c`
`infile`	Path to input file (from `-i`)
`outfile`	Path to output file (from `-o`)
`use_model`	`true` when `-m` is present (compression only)
`adaptive_scan`	`true` when `-a` is present (compression only)
`width`	Image width in pixels from `-w`; default `-1` (invalid sentinel)

Important: `use_model`, `adaptive_scan`, and `width` are only meaningful during compression. During decompression the codec ignores them entirely and reads the equivalent information from the compressed file header (flags byte and width/height fields). This design ensures the decompressor is self-contained.

3.4 parse_arguments()

`ParsedArgs parse_arguments(int argc, char** argv)` is the only exported function. It performs the following steps:

1. Registers all flags and options with `argparse.hpp`.
2. Calls the library's `parse` method against `argc`/`argv`.
3. Extracts values into a `ParsedArgs` instance.
4. Validates mode consistency:
   - Exactly one of `-c` or `-d` must be present.
   - `-i` and `-o` are required in both modes.
   - In compression mode (`-c`), `-w` must be provided and positive.
5. On any validation failure: prints a usage message to `stderr` and calls `exit(1)`.

3.5 argparse.hpp

`argparse.hpp` is a third-party header-only C++ argument parsing library bundled directly in `include/`. It handles option registration, type conversion, and help text generation.

It is **not** a compression library and does not violate the assignment's "no external compression libraries" constraint. Using it avoids reimplementing `getopt`-style parsing and reduces `args.cpp` to ~30 lines of straightforward setup code.

4 BitIO Module

4.1 Purpose

`bitio.hpp` provides a pair of thin wrappers around `std::ostream` and `std::istream` that allow individual bits or groups of bits to be written and read. It is the lowest-level primitive in the codebase: all LZSS token emission and decoding goes through `BitWriter` and `BitReader`.

Without this abstraction, LZSS would need to manage its own bit packing — mixing algorithm logic with serialization detail. By delegating that concern to a dedicated class, `lzss.cpp` can express its logic in clean terms (`write_bits(offset, 12)`) with no manual shifting.

4.2 Files

`include/bitio.hpp` only. Both classes are fully defined in the header (all methods are short and inlineable), so no `.cpp` file is needed. This keeps the build simple: any translation unit that includes `bitio.hpp` gets the full implementation at zero link cost.

4.3 BitWriter

`BitWriter` buffers bits until a full byte is available, then emits it to the underlying stream.

```
1  class BitWriter {
2  public:
3      explicit BitWriter(std::ostream& out);
4      void write_bit(bool b);
5      void write_bits(uint32_t val, int n);  // n LSBs of val, MSB
           first
6      void flush();
7  private:
8      std::ostream& out_;
9      uint8_t buf_;          // accumulation byte (bits fill from
           MSB down)
10     int bits_in_buf_;      // 0..7: bits currently in buf_
11 };
```

State `buf_` accumulates bits left-to-right (MSB first). `bits_in_buf_` counts how many bits have been placed in `buf_` since the last byte was emitted.

write_bit(b) Shifts `buf_` left by one and ORs the new bit into the LSB position, then increments `bits_in_buf_`. When the counter reaches 8 the byte is emitted and both fields are reset.

write_bits(val, n) Iterates from bit `n-1` down to 0, calling `write_bit` for each. This writes the `n` least-significant bits of `val` with the most-significant of those bits first (big-endian bit order within the group).

flush() If any bits remain in `buf_` (`bits_in_buf_` > 0), the partial byte is left-shifted to fill the remaining positions with zero bits, then emitted. **The caller must call `flush()` after the last write.** Forgetting this call silently drops the final partial byte.

Example: building byte `0xB0` Four calls `write_bit(1)`, `write_bit(0)`, `write_bit(1)`, `write_bit(1)` build `buf_` to `0b00001011` (4 bits, pending). A subsequent `flush()` left-shifts by 4 and emits `0xB0 = 0b10110000`.

After call	`buf_` (binary)	`bits_in_buf_`
write_bit(1)	`00000001`	1
write_bit(0)	`00000010`	2
write_bit(1)	`00000101`	3
write_bit(1)	`00001011`	4
flush()	emits `0xB0` (`10110000`)	0

4.4 BitReader

`BitReader` fetches bytes from the stream on demand and returns bits one at a time.

```
class BitReader {
public:
    explicit BitReader(std::istream& in);
    bool read_bit();
    uint32_t read_bits(int n);  // reads n bits MSB-first
    bool eof() const;
private:
    std::istream& in_;
    uint8_t buf_;      // current byte being consumed
    int bits_left_;    // how many bits remain in buf_
};
```

read_bit() When `bits_left_` reaches 0, the next byte is fetched from the stream into `buf_` and `bits_left_` is set to 8. The most-significant remaining bit is extracted as `(buf_ >> bits_left_) & 1`, and `bits_left_` is decremented. On stream EOF the method returns `false`.

read_bits(n) Accumulates `n` calls to `read_bit()` into a `uint32_t`, shifting the accumulator left after each bit. This reconstructs the MSB-first value that `write_bits` encoded.

eof() Returns `true` when the underlying stream is exhausted and no bits remain in `buf_`.

4.5 Design Notes

No ownership. Both classes hold a *reference* to the stream, not a pointer or `unique_ptr`. The caller owns and manages the stream's lifetime; BitWriter/Reader are lightweight view objects that must not outlive the stream they wrap.

Header-only. All methods are short enough that the inline cost is negligible. Keeping both classes in a single header avoids an extra translation unit and simplifies the build.

5 Delta Model Module

5.1 Purpose

The model module applies a first-order differential (delta) transform to a byte stream *before* LZSS encoding, and its exact inverse *after* LZSS decoding.

In a smooth grayscale image, adjacent pixels differ by only a few intensity levels. The raw pixel stream therefore contains values distributed across the full 0–255 range. After delta coding, the stream consists mostly of small values near zero (e.g., $-3, 0, +2, 0, +1, \ldots$), which means many bytes repeat the same small value. LZSS finds longer matches in such a stream, producing a higher compression ratio.

The effect is strongest for images with gradual gradients and least effective for high-frequency noise images.

5.2 Files

- `include/model.hpp` — function declarations.
- `src/model.cpp` — implementations (~20 lines total).

5.3 forward_model()

```
void forward_model(std::vector<uint8_t>& data);
```

Effect: transforms `data` in-place so that `data`$[i]$ = `data`$[i]$ − `data`$[i-1]$ (unsigned 8-bit arithmetic) for all $i > 0$. `data[0]` is left unchanged (it is the anchor value).

Loop direction — backward:

```
for (size_t i = data.size() - 1; i > 0; --i)
    data[i] = (uint8_t)(data[i] - data[i - 1]);
```

The loop iterates *from the end toward the start*. This is essential: when computing `data[i]`, we need the *original* `data[i-1]`, not the already-overwritten delta value. Iterating backward ensures `data[i-1]` has not yet been modified.

A forward loop would require a temporary variable to preserve the previous original value. The backward loop achieves the same result with no extra storage.

Arithmetic: The subtraction uses `uint8_t` (unsigned 8-bit) arithmetic. Subtraction wraps modulo 256 on all platforms without undefined behaviour. Values of the original stream are in 0–255; differences can be anywhere in 0–255 when interpreted as unsigned (e.g., $3 - 200 = 59$ in unsigned 8-bit arithmetic).

5.4 inverse_model()

```
void inverse_model(std::vector<uint8_t>& data);
```

Effect: reconstructs the original values from a delta-coded buffer. For all $i > 0$: `data`$[i] =$ `data`$[i] +$ `data`$[i-1]$. `data[0]` is unchanged.

Loop direction — forward:

```
for (size_t i = 1; i < data.size(); ++i)
    data[i] = (uint8_t)(data[i] + data[i - 1]);
```

Here the loop must iterate *forward*: to reconstruct `data[i]`, we need the already-reconstructed `data[i-1]` (its original value, now restored). Using the reconstructed prefix accumulates the cumulative sum correctly (prefix-sum / running total).

5.5 Round-Trip Guarantee

`inverse_model(forward_model(`$\mathbf{x}$`))` $= \mathbf{x}$ for all inputs $\mathbf{x}$.

This holds because modular arithmetic satisfies $(a - b) + b \equiv a \pmod{256}$. No floating-point rounding or overflow is possible.

5.6 Integration with the Codec

`forward_model` is called inside `codec.cpp` after scanning (scan gives the 1-D byte stream) and before LZSS encoding. `inverse_model` is called after LZSS decoding and before unscanning.

In adaptive mode, the model is applied per-block (each block is scanned independently, then modelled independently). The first pixel of each block serves as its own anchor; there is no cross-block delta dependency.

6 Scanner Module

6.1 Purpose

The scanner module converts the 2-D pixel array of a grayscale image into 1-D byte streams suitable for LZSS, and performs the inverse operation during decompression. It also handles block decomposition for adaptive mode.

The motivation is that LZSS operates on a linear byte sequence. The order in which pixels are serialised affects how much local repetition (and therefore compressibility) the 1-D stream contains.

6.2 Files

- `include/scanner.hpp` — struct definitions and function declarations.
- `src/scanner.cpp` — implementations (~130 lines).

6.3 Data Structures

6.3.1 BlockMeta

```
struct BlockMeta {
    uint8_t scan_dir;    // 0 = horizontal, 1 = vertical
    uint8_t compressed;  // 1 = stored as LZSS, 0 = stored raw
};
```

Two bits of per-block metadata stored in the compressed file. `scan_dir` tells the decompressor which unscan function to call. `compressed` tells it whether to run `lzss_decode` or copy bytes directly.

6.3.2 Block

```
struct Block {
    std::vector<uint8_t> pixels;  // row-major within the block
    uint32_t block_row;           // row index in the image grid
       (0-based)
    uint32_t block_col;           // column index in the image
       grid (0-based)
    uint32_t width;               // actual block width  (<=
       block_size at right edge)
    uint32_t height;              // actual block height (<=
       block_size at bottom edge)
    BlockMeta meta;               // set during compression
};
```

`pixels` is stored in row-major order within the block (independent of whether the block will ultimately be encoded with a horizontal or vertical scan). `block_row` and `block_col` record the block's position in the image grid so `merge_blocks` can place pixels back in the correct location. `width` and `height` may be smaller than `block_size` for edge blocks when the image dimensions are not exact multiples of `block_size`.

6.4 Scan Functions

6.4.1 scan_horizontal / unscan_horizontal

```
std::vector<uint8_t> scan_horizontal(
    const std::vector<uint8_t>& pixels, uint32_t width, uint32_t
       height);
std::vector<uint8_t> unscan_horizontal(
    const std::vector<uint8_t>& data,   uint32_t width, uint32_t
       height);
```

A horizontal scan visits pixels left-to-right, top-to-bottom (row-major order). Since `pixels` is already stored in row-major order, `scan_horizontal` is a trivial copy of the input. `unscan_horizontal` is identical. The `width` and `height` parameters are accepted for API symmetry but are unused (annotated `/*width*/, /*height*/` in the source).

Note: This means `scan_horizontal` incurs a `vector<uint8_t>` heap allocation and copy that could be skipped with a minor `codec.cpp` branch (see the Improvements chapter).

6.4.2 scan_vertical / unscan_vertical

```
std::vector<uint8_t> scan_vertical(
    const std::vector<uint8_t>& pixels, uint32_t width, uint32_t
       height);
std::vector<uint8_t> unscan_vertical(
    const std::vector<uint8_t>& data,   uint32_t width, uint32_t
       height);
```

A vertical scan visits pixels top-to-bottom, left-to-right (column-major order): for each column c from 0 to `width` $-$ 1, emit all rows from top to bottom.

`scan_vertical` implementation:

```
for (uint32_t col = 0; col < width; ++col)
    for (uint32_t row = 0; row < height; ++row)
        out.push_back(pixels[row * width + col]);
```

`unscan_vertical` is the inverse: the input data is column-major, so column c occupies positions $[c \cdot$ `height`, $(c+1) \cdot$ `height`$)$, and each element maps back to `result[row * width + col]`:

```
for (uint32_t col = 0; col < width; ++col)
    for (uint32_t row = 0; row < height; ++row)
        result[row * width + col] = data[col * height + row];
```

6.5 Block Functions

6.5.1 split_into_blocks

```
std::vector<Block> split_into_blocks(
    const std::vector<uint8_t>& pixels,
    uint32_t width, uint32_t height, uint32_t block_size);
```

Divides the image into a grid of `block_size`×`block_size` tiles in raster order (left-to-right, top-to-bottom). The total number of blocks is $\lceil$`width/block_size`$\rceil \times \lceil$`height/block_size`$\rceil$.

For each block at grid position (`br`, `bc`):

- `width` = min(`block_size`, `image_width` $-$ `bc` $\cdot$ `block_size`)
- `height` = min(`block_size`, `image_height` $-$ `br` $\cdot$ `block_size`)
- `pixels` is extracted row-by-row from the source image.
- Default `meta` = `{scan_dir=0, compressed=1}` (assume H-scan and compressed until proven otherwise).

6.5.2 merge_blocks

```
std::vector<uint8_t> merge_blocks(
    const std::vector<Block>& blocks,
    uint32_t width, uint32_t height, uint32_t block_size);
```

Reconstructs the full image from a vector of blocks. Each block already carries `block_row` and `block_col` coordinates, so the function places each pixel at its correct position in the output buffer. This is called after all blocks have been decompressed and unscanned.

Note: `merge_blocks` is only used by the decompressor in adaptive mode. The compressor works directly with the `Block` structs and does not need to reassemble the image.

6.6 SAD Heuristics

```
1 uint32_t compute_sad_horizontal(const std::vector<uint8_t>& block
     , uint32_t w, uint32_t h);
2 uint32_t compute_sad_vertical  (const std::vector<uint8_t>& block
     , uint32_t w, uint32_t h);
```

Sum of Absolute Differences (SAD) measures how smooth a block is in each direction.

`compute_sad_horizontal` sums $|\texttt{block}[r \cdot w + c] - \texttt{block}[r \cdot w + c - 1]|$ for all rows r and columns $c > 0$. A small horizontal SAD means adjacent pixels in the same row differ little: the horizontal scan direction will expose long repeating runs to LZSS.

`compute_sad_vertical` sums $|\texttt{block}[r \cdot w + c] - \texttt{block}[(r-1) \cdot w + c]|$ for all columns c and rows $r > 0$. A small vertical SAD means adjacent pixels in the same column differ little.

Decision rule (implemented in `codec.cpp`): choose the direction with the *lower* SAD. Lower SAD $\Rightarrow$ smoother transitions in that direction $\Rightarrow$ more repetition in the resulting 1-D stream $\Rightarrow$ better LZSS compression.

7 LZSS Encoder/Decoder

7.1 Purpose

LZSS is the core compression algorithm. It scans a 1-D byte stream and replaces repeated sequences with compact back-references into a sliding history window, emitting only the tokens and flag bits needed by the decoder to reconstruct the original stream.

All other modules (scanner, model) are pre-processors that rearrange or transform the data to increase the regularity that LZSS then exploits.

7.2 Files

- `include/lzss.hpp` — constants and public API declarations.
- `src/lzss.cpp` — private data structures and implementations (~230 lines).

7.3 Constants

Constant	Value	Rationale
LZSS_WINDOW_SIZE	4096	12-bit offset; one row of a 256-wide image = 256 B, 16 rows fit
LZSS_LOOKAHEAD	32	5-bit length field (values 0..31 = actual lengths 3..34 bytes)
LZSS_MIN_MATCH	3	Match token costs 18 bits; 3 literals cost 27 bits — net saving at 3
LZSS_OFFSET_BITS	12	Token: 12 bits for the window offset
LZSS_LENGTH_BITS	5	Token: 5 bits for adjusted length (length − MIN_MATCH)

MIN_MATCH derivation: A match token costs $1 + 12 + 5 = 18$ bits. Three literals each cost $1 + 8 = 9$ bits, totalling 27 bits. A match of length 3 therefore saves 9 bits. A match of length 2 would cost the same as 2 literals (18 bits vs $2 \times 9 = 18$ bits) — no saving — so 3 is the minimum meaningful match length.

7.4 Internal Data Structures

7.4.1 Window

```
struct Window {
    uint8_t  buf[LZSS_WINDOW_SIZE] = {};
    uint32_t pos = 0;

    void push(uint8_t byte) {
        buf[pos] = byte;
        pos = (pos + 1) % LZSS_WINDOW_SIZE;
    }
};
```

A circular byte buffer of 4096 bytes representing the sliding history. `pos` always points to the slot that will receive the *next* byte. Both encoder and decoder maintain an identical `Window` instance so that the decoder can reconstruct matches from its own copy of history without any explicit dictionary transmission.

7.4.2 FlagGroup

```
struct FlagGroup {
    static constexpr int GROUP = 8;
    uint8_t  flags[8];
    uint32_t offsets[8];   // match: window offset
    uint32_t lengths[8];   // match: length - LZSS_MIN_MATCH (0-
       based)
    uint8_t  literals[8];  // literal byte
    int count = 0;         // tokens accumulated (0..8)

    void add_literal(uint8_t byte);
    void add_match(uint32_t offset, uint32_t length);
    bool full() const;
    void flush(BitWriter& out);
};
```

Collects up to 8 tokens before emitting them as a group. When `flush()` is called:

1. One *flag byte* is emitted, with each bit (MSB first) indicating whether the corresponding token is a match (1) or a literal (0). If the group is partial (fewer than 8 tokens), the remaining bits are padded with 0.
2. Each token's payload is emitted in order: literal → 8 bits; match → 12-bit offset + 5-bit adjusted length.

The length stored in the token is `length − LZSS_MIN_MATCH`, so a 3-byte match stores 0 and a 34-byte match stores 31. The decoder adds `LZSS_MIN_MATCH` back during decoding.

7.4.3 HashChains

```
struct HashChains {
    static constexpr uint32_t HASH_SIZE = 65536;
    static constexpr uint32_t NIL = LZSS_WINDOW_SIZE;  // sentinel

    uint32_t head[65536];           // head[h] = most-recent window slot with hash h
    uint32_t prev[LZSS_WINDOW_SIZE]; // prev[slot] = previous slot with same hash

    static uint32_t hash2(uint8_t a, uint8_t b); // (a << 8) | b
    void insert(uint32_t slot, uint8_t a, uint8_t b);
    std::pair<uint32_t,uint32_t> find_match(...);
};
```

A hash-based match finder using linked lists within the sliding window.

Hash function: `hash2(a, b) = (a ≪ 8) | b` produces a 16-bit key from the first two bytes of a candidate position. The 65536-bucket hash table maps each 2-byte pair to the most recent window slot that starts with those two bytes.

insert(slot, a, b): prepends `slot` to the chain for hash `(a,b)`. The existing head becomes `prev[slot]`.

find_match(pattern, len, win): walks the chain for `hash2(pattern[0], pattern[1])`, comparing up to `LZSS_LOOKAHEAD` bytes at each candidate position. The chain walk is capped at 128 iterations (`chain_limit`) to bound worst-case encoder cost on pathological inputs.

Matches shorter than `LZSS_MIN_MATCH` are discarded; the function returns `(0, 0)` in that case. Otherwise it returns the `(offset, length)` pair for the longest match found.

7.5 lzss_encode()

```
void lzss_encode(const std::vector<uint8_t>& input, BitWriter& out);
```

Delegates to the private `encode_impl()` function, which:

1. Initialises `Window`, `HashChains`, `FlagGroup`, position `pos=0`.
2. While `pos < input.size()`:

(a) Calls `HashChains::find_match(input + pos, lookahead, win)`.

(b) If length $\geq$ `MIN_MATCH`: `FlagGroup::add_match`; advance `pos` by `length`; push each consumed byte to the window and insert its hash.

(c) Else: `FlagGroup::add_literal(input[pos])`; advance by 1; push and insert.

(d) If `FlagGroup::full()`: `FlagGroup::flush(out)`.

3. Flush the final (partial) group.

The caller is responsible for calling `BitWriter::flush()` after `lzss_encode()` returns.

7.6 lzss_encode_to_buffer()

```
std::vector<uint8_t> lzss_encode_to_buffer(const std::vector<
    uint8_t>& input);
```

Wraps `encode_impl` with a `std::ostringstream` to collect output in memory. After encoding, extracts the string into a `vector<uint8_t>`. Used by `codec.cpp` in adaptive mode to check compressed size before committing to the output file (see Section 8.5).

7.7 lzss_decode()

```
void lzss_decode(BitReader& in, std::vector<uint8_t>& output,
    size_t expected_size);
```

Reconstructs exactly `expected_size` bytes from the compressed bit stream:

1. Initialises `Window`, clears and reserves `output`.
2. While `output.size() < expected_size`:
   (a) Reads one 8-bit flag byte.
   (b) For each bit i from 7 down to 0 (MSB first), while bytes still needed:
   - Bit = 0 (literal): read 8-bit value; append to `output`; push to window.
   - Bit = 1 (match): read 12-bit offset, 5-bit adjusted length. Actual length = adjusted + `MIN_MATCH`. Copy `length` bytes from the window at distance `offset`, appending each byte to `output` and pushing it back to the window. The copy handles overlapping matches (where `length > offset`) by clamping `effective_k = k % offset` — this reproduces the run-length extension behaviour of the encoder.

The decoder stops as soon as `output.size() == expected_size`, even if the flag byte's remaining bits have not been consumed. A partial flag group at the end of the stream is therefore normal and expected.

8 Codec Module

8.1 Purpose

`codec.cpp` is the top-level orchestration layer. It owns the LZKO file format, drives all other modules in the correct order, and implements both the static and adaptive compression strategies. The public interface (`codec.hpp`) exposes exactly two functions: `compress()` and `decompress()`.

8.2 Files

- `include/codec.hpp` — public declarations.
- `src/codec.cpp` — file format + all compression/decompression logic (~200 lines).

8.3 Private Constants and Helpers

```
1 static constexpr uint32_t BLOCK_SIZE     = 16;
2 static constexpr uint8_t  BLOCK_SIZE_LOG = 4;    // 2^4 = 16
```

`BLOCK_SIZE` is defined here and not in `scanner.hpp` (see the Improvements chapter for a discussion of this choice).

Four little-endian I/O helpers are defined as file-private (`static`) functions:

```
1 static void     write_u16(std::ostream&, uint16_t);
2 static void     write_u32(std::ostream&, uint32_t);
3 static uint16_t read_u16(std::istream&);
4 static uint32_t read_u32(std::istream&);
```

Little-endian byte order is used throughout because the primary target platform (merlin.fit.vutbr.cz, x86-64) is natively little-endian and this matches the internal `uint16_t`/`uint32_t` layout without byte swapping.

8.3.1 pack_block_meta / unpack_block_meta

```
1 static std::vector<uint8_t> pack_block_meta(const std::vector<
    Block>&);
2 static void unpack_block_meta(const std::vector<uint8_t>& buf,
    uint32_t n,
3                               std::vector<uint8_t>& scan_dirs,
4                               std::vector<uint8_t>& comp_flags);
```

These functions pack two bits per block into a byte array (or unpack in the reverse direction). Block i occupies bit positions $2i$ (scan_dir) and $2i+1$ (compressed), numbered MSB-first within each byte.

Bit address formulae:

- Byte index: $\lfloor 2i/8 \rfloor = \lfloor i/4 \rfloor$
- Bit position within byte (0 = MSB): $(2i) \bmod 8$
- Mask: `1u << (7 - bit_position)`

8.4 File Format

Offset	Size	Field
0	4 B	Magic: `0x4C 0x5A 0x4B 0x4F` ("LZKO")
4	2 B	`uint16_t` width (little-endian)
6	2 B	`uint16_t` height (little-endian)
8	1 B	Flags: bit 0 = `use_model`, bit 1 = `adaptive`
9	1 B	`block_size_log2` (= 4)
10	4 B	`uint32_t` original pixel count
14	...	Static: LZSS bit-stream
14	...	Adaptive: `uint32_t num_blocks` + metadata + per-block data

Adaptive per-block data:

- Packed 2-bit metadata: $\lceil 2 \cdot n/8 \rceil$ bytes, MSB-first.
- Followed by n entries, each: `uint32_t stored_size` + `stored_size` bytes (LZSS-compressed or raw pixels).

8.5 compress()

```
int compress(std::istream& in, std::ostream& out,
             int width, bool use_model, bool adaptive);
```

Step 1: Load and validate. All bytes from `in` are read into `vector<uint8_t> pixels`. If `pixels.size() % width != 0`, an error is printed and 1 is returned.

Step 2: Write header. Magic, width, height, flags byte (`use_model | (adaptive << 1)`), `BLOCK_SIZE_LOG`, and `pixels.size()` are written.

Step 3a: Static path (`adaptive == false`):

1. `scan_horizontal(pixels, w, h)` → `data` (trivial copy).
2. If `use_model`: `forward_model(data)`.
3. `lzss_encode(data, bw)`; `bw.flush()`.

Step 3b: Adaptive path (`adaptive == true`), two passes:

Pass 1 — metadata determination:

1. `split_into_blocks(pixels, w, h, BLOCK_SIZE)` → `blocks`.
2. For each block:
   - Choose `scan_dir` by comparing `compute_sad_horizontal` vs `compute_sad_vertical`.
   - Scan the block pixels accordingly.
   - If `use_model`: `forward_model(scanned)`.
   - `lzss_encode_to_buffer(scanned)` → `encoded`.
   - `meta.compressed = (encoded.size() < scanned.size()) ? 1 : 0`.

Write metadata: `num_blocks` + `pack_block_meta(blocks)`.

Pass 2 — data output:

1. For each block: repeat scan + model. If `meta.compressed`: `lzss_encode_to_buffer` → write size + bytes. Else: write size + raw scanned bytes.

Two-pass rationale: The file format requires all block metadata to precede block data. Because the metadata (specifically `compressed`) depends on the encoded size, which is only known after encoding, the codec must encode each block once to determine metadata, write all metadata, then encode again to write the data. This doubles the LZSS encoding work for adaptive mode. See the Improvements chapter for a fix that caches the encoded bytes from Pass 1.

8.6 decompress()

```
1 int decompress(std::istream& in, std::ostream& out);
```

Step 1: Validate magic. Reads 4 bytes; if they do not match `0x4C 0x5A 0x4B 0x4F`, prints an error and returns 1 immediately.

Step 2: Read header. Extracts `width`, `height`, `flags` (from which `use_model` and `adaptive` are derived), `block_size` ($= 2^{\texttt{block\_log}}$), and `original_size`.

Step 3a: Static path:

1. `lzss_decode(BitReader(in), data, original_size)`.
2. If `use_model`: `inverse_model(data)`.
3. Write `data` to `out`.

Step 3b: Adaptive path:

1. Read `num_blocks`, then $\lceil 2 \cdot \texttt{num\_blocks}/8 \rceil$ metadata bytes.
2. `unpack_block_meta` → `scan_dirs[]`, `comp_flags[]`.
3. Allocate `image(width * height)`.
4. For each block index `idx`:
   - Compute `block_row = idx / blocks_x`, `block_col = idx % blocks_x`, actual `bw`, `bh`.
   - Read `stored_size` + raw bytes into `block_raw`.
   - If `comp_flags[idx]`: wrap in `istringstream` + `BitReader` + `lzss_decode(brdr, data, bw*bh)`. Else: `data = block_raw`.
   - If `use_model`: `inverse_model(data)`.
   - `unscan_horizontal` or `unscan_vertical` based on `scan_dirs[idx]`.
   - Copy unscanned block pixels into the correct position in `image`.
5. Write `image` to `out`.

9 Entry Point (main.cpp)

9.1 Purpose

`src/main.cpp` is a minimal entry point. It contains no business logic: it delegates argument parsing to the `args` module and all compression/decompression work to the `codec` module. Keeping it thin means the codec is independently testable — it only requires `std::istream` and `std::ostream` references, not real files.

9.2 Implementation

The function performs exactly four logical steps:

1. **Parse arguments:** `ParsedArgs args = parse_arguments(argc, argv)`. On invalid arguments, `parse_arguments` prints a usage message and exits; `main` never sees an invalid `ParsedArgs`.
2. **Open streams:** `std::ifstream in(args.infile, std::ios::binary)` and `std::ofstream out(args.outfile, std::ios::binary)`. Both are checked immediately; if either fails, an error is printed to `std::cerr` and `main` returns 1.
3. **Dispatch:** `args.decompress` → `decompress(in, out)` or `compress(in, out, args.width, args.use_model, args.adaptive_scan)`.
4. **Return:** the exit code from the called function (0 = success, 1 = error).

9.3 DEBUG Macro

When compiled with `-DDEBUG`, `main.cpp` prints mode, input/output paths, image width, and the model/scan flags to `std::cout` before dispatching. This output is conditionally compiled:

```
#ifdef DEBUG
    std::cout << "Mode: " << (args.decompress ? "decompress" : "
       compress") << "\n";
    // ...
#endif
```

No debug output is produced in release builds. The `Makefile` does not define `DEBUG` by default.

9.4 Error Messages: Czech vs English

Several error strings in `main.cpp` and `codec.cpp` are currently in Czech (e.g., *"Nelze otevřít vstupní soubor:"*, *"Neplatný formát souboru"*). The assignment requires English *comments* in source files; error messages to `stderr` are not comments and are therefore technically not in scope of that rule. However, mixing Czech runtime output with an otherwise English-commented codebase is inconsistent. See the Improvements chapter for the recommended fix.

10 Improvements and Simplification Analysis

The codebase is lean and well-structured for its scope: approximately 800 lines across 8 source files, with clear single-responsibility boundaries. No files are unnecessary, and the module dependency graph is acyclic and logical.

The items below are *opportunities*, not defects — the code is correct and all round-trips are lossless. Each item is labelled with a category and a priority.

10.1 Critical / High-Priority Improvements

10.1.1 Double Encoding in Adaptive Mode

Category: Performance. Priority: High. Effort: Low.

In `compress()` (adaptive path), Pass 1 calls `lzss_encode_to_buffer()` for every block to obtain the compressed size and determine whether to set `meta.compressed = 1`. Pass 2 then re-encodes the *same* block for actual output.

For a 512×512 image divided into 16×16 blocks, there are 1024 blocks, so the encoder runs 2048 times instead of 1024. Adaptive mode is therefore roughly twice as slow as necessary.

Root cause: the file format requires all block metadata to precede block data (so the decompressor knows the layout before reading any data). A naive single-pass encoder cannot write metadata first without knowing sizes upfront.

Fix: in Pass 1, store the encoded bytes — not just the size — alongside `meta.compressed`. One option is to extend `Block` with a `vector<uint8_t> encoded_data` field populated during Pass 1 and consumed (then freed) during Pass 2. Another option is a parallel `vector<vector<uint8_t>>` alongside `blocks`.

10.1.2 No Unit Tests

Category: Quality / Maintainability. Priority: High. Effort: Medium.

There is no unit test infrastructure. Round-trip correctness is only verified by `make test`, which exercises the full pipeline on whole image files. A bug in `BitWriter::flush()`, `lzss_decode()`, or `inverse_model()` would surface only as a corrupted output file, not as a targeted test failure pointing directly to the broken function.

Fix: add a `tests/test_main.cpp` (or similar) that directly invokes:

- `BitWriter`/`BitReader` round-trip for edge cases (empty, 1 bit, non-multiple of 8 bits).
- `forward_model` / `inverse_model` round-trip on ramp, flat, and random data.
- `scan_horizontal`/`unscan_horizontal` and `scan_vertical`/`unscan_vertical` inverses.
- `lzss_encode` / `lzss_decode` round-trip on all-same, alternating, and random byte sequences.

No test framework is required; a standalone `main` with assertions is sufficient.

10.2 Quality / Correctness Improvements

10.2.1 Czech Error Messages

Category: Quality. Priority: Medium. Effort: Low.

Three user-visible strings use Czech:

- `main.cpp`: *"Nelze otevřít vstupní soubor:"* and *"Nelze otevřít výstupní soubor:"*
- `codec.cpp`: *"Neplatný formát souboru (chybné magic bytes)"* and *"Velikost vstupu ... není dělitelná šířkou"*

The assignment requires English *comments*; error messages are not comments. However, mixing Czech runtime output with an otherwise English-commented codebase is inconsistent and could confuse future readers.

Fix: translate all four strings to English equivalents such as *"Cannot open input file:"*, *"Invalid file format (bad magic bytes)"*, etc.

10.2.2 Width Validation Belongs in Argument Parsing

Category: Correctness / Defensive Programming. Priority: Medium. Effort: Low.

`args.width` defaults to -1 as a sentinel for "not provided". When passed as an `int` to `compress()`, which immediately converts it to `uint32_t`, a value of -1 becomes $2^{32} - 1 = 4{,}294{,}967{,}295$. The subsequent check `pixels.size() % image_width != 0` may technically avoid a crash but produces a misleading error message about image dimensions.

Fix: validate `width > 0` inside `parse_arguments()` (in compression mode) and exit immediately with a clear message such as *"Error: -w must be a positive integer."*. This removes an implicit semantic dependency between `args.hpp` and `codec.cpp` around the -1 sentinel.

10.3 Minor / Low-Priority Improvements

10.3.1 scan_horizontal Is an Unnecessary Heap Allocation

Category: Performance / Simplification. Priority: Medium. Effort: Low.

`scan_horizontal()` returns a copy of its input unchanged (row-major storage is the native layout). For a 512×512 image in static mode this allocates 262 144 bytes unnecessarily. In adaptive mode, every block choosing horizontal scan incurs the same cost.

Options:

1. Add a branch in `codec.cpp` to skip the scan call for horizontal direction and operate directly on the source pixels.
2. Make `scan_horizontal` a no-op that returns the input by move (if the caller passes by value) or provide a separate `scan_horizontal_inplace` that returns a `const&`.

Option 1 is simpler. The API symmetry between `scan_horizontal` and `scan_vertical` is worth keeping for clarity, but the unnecessary copy in the H-path is easy to eliminate.

10.3.2 lzss_encode_to_buffer and ostringstream Overhead

Category: Performance. Priority: Low. Effort: Medium.

`lzss_encode_to_buffer` wraps a `std::ostringstream` to collect output bytes, then copies the internal string buffer into a `vector<uint8_t>`. This involves two separate heap allocations.

Since each block is at most 256 bytes (16×16) and the LZSS-encoded output is similarly bounded, the absolute cost is small. A `vector`-backed `streambuf` would eliminate one allocation, but this optimisation is unlikely to be noticeable in practice.

10.3.3 BLOCK_SIZE Defined in codec.cpp Only

Category: Simplification / Cohesion. Priority: Low. Effort: Low.

`BLOCK_SIZE = 16` and `BLOCK_SIZE_LOG = 4` are defined as `static constexpr` in `codec.cpp`, making them invisible to other modules. Logically, block size belongs with the block-related types (`Block`, `BlockMeta`) in `scanner.hpp`.

Fix: move both constants to `scanner.hpp` so any future component that needs to know the block size can find it in the obvious place.

10.4 What Should Not Be Changed

The following aspects of the current design are correct and should be preserved:

- **7-file module structure.** `args`, `bitio`, `lzss`, `model`, `scanner`, `codec`, `main` each have a single clear responsibility. Do not collapse `scanner` and `model` into `codec` — the separation aids readability and testability.
- **LZSS sliding-window algorithm.** LZSS is the right algorithm for this problem: it exploits spatial correlation in images, requires no stored dictionary, and has a trivially simple decoder. There is no reason to switch to LZ78/LZW or a statistical method.
- **Flag-group encoding (8 tokens per flag byte).** The current byte-aligned flag packing is a standard LZSS practice. It keeps output byte-aligned and is easy to decode with a simple bitmask loop.
- **argparse.hpp inclusion.** The third-party argument parsing library is not a compression library and does not violate the assignment constraint. Replacing it with manual `getopt` parsing would save a few hundred lines of header but add maintenance burden with no benefit.
- **Header-only BitIO.** `BitWriter` and `BitReader` are short enough to be inline. A separate `bitio.cpp` translation unit would add build complexity for no benefit.

10.5 Summary Table

#	Item	Category	Priority	Effort
1	Double encoding in adaptive mode	Performance	High	Low
2	No unit tests	Quality	High	Medium
3	Czech error messages	Quality	Medium	Low
4	Width validation in argument parser	Correctness	Medium	Low
5	scan_horizontal unnecessary copy	Performance	Medium	Low
6	ostringstream in encode_to_buffer	Performance	Low	Medium
7	BLOCK_SIZE defined in codec.cpp	Simplification	Low	Low